

COMS 3110: Homework 5
Due: December 6th, 11:59pm
Total Points: 60

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions will have to be typed. You can use word, LaTeX, or any other type-setting tool to type your solution. Your submission file should be in pdf format. Do **NOT** submit a photocopy of handwritten homework except for diagrams that can be hand-drawn and scanned. We reserve the right **NOT** to grade homework that does not follow the formatting requirements. Name your submission file: `<Your-net-id>-3110-hw5.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw5.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed).

General Requirements

- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Some Useful (in)equalities

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
 - $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
 - $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
 - $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
 - $a + ar + ar^2 + \dots + ar^{n-1} = \frac{a(r^n - 1)}{r - 1}$
 - $1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
 - $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$
-

1. (10 pts) You are helping a group of ethnographers analyze some oral history data. The ethnographers interviewed members of a village and learned about the birth and death a set of n people: $P_1, P_2, \dots, P_n$. The information is categorized as follows:

- For some i and j , person P_i died before person P_j was born
- For some i and j , the life spans of P_i and P_j overlapped at least partially.

As these are word of mouth information, it becomes important to validate the data. You are tasked with writing an algorithm which takes as input the learned information (about relative ordering of birth and death of n people) and verify its validity.

- Construct a graph as follows: For all i , construct a vertex b_i to represent birth of P_i and construct another vertex d_i to represent death of P_i . Add directed edge from b_i to d_i . The edge captures the fact that for any person the birth occurs before death.
- Next, based on the information gathered if P_i died before P_j is born, then add a directed edge from d_i to b_j .
- Finally, if P_i and P_j have overlapping lifespan, then add a directed edge from b_i to d_j and another directed edge from b_j to d_i . These edges capture the fact that these persons have overlapping lifespan, i.e., P_i was born before P_j 's death and similarly, P_j was born before P_i 's death.
- Now, we have a graph that represents the temporal ordering of the birth and death events of all people. For the information to be valid, the graph must be a DAG (directed acyclic graph). DFS explore the entire graph and look for backedges. If no backedge exists, then the graph is a DAG and the information is valid; otherwise the information is invalid.
- Overall runtime is $O(V + E)$, where V is two times the number of persons (for each person we have two vertices) and the number of edges is bounded by the sum of the number of people and the number of information pairs.

2. (10 pts) Solve the following recurrence equations using the recurrence tree method. Show the table as done in lecture.

(a) (5 pts)

$$T(n) = \begin{cases} 1 & \text{if } n < 2 \\ 4 \cdot T(\frac{n}{2}) + n & \text{if } n \geq 2 \end{cases}$$

Level	Problem Size	# Nodes	Work / Node	Level Total
1	$\frac{n}{2^0}$	$4^0 = 1$	n	n
2	$\frac{n}{2^1}$	4^1	$\frac{n}{2^1}$	$2^1 n$
...	...	...	...	...
i	$\frac{n}{2^{i-1}}$	4^{i-1}	$\frac{n}{2^{i-1}}$	$2^{i-1} n$

The level total is computed by multiplying the number of nodes with the work per node for that level:

$$\begin{aligned}
4^{i-1} \times \frac{n}{2^{i-1}} &= 2^{2(i-1)} \times \frac{n}{2^{i-1}} \\
&= 2^{i-1} \times 2^{i-1} \times \frac{n}{2^{i-1}} \\
&= 2^{i-1} \cdot n
\end{aligned}$$

Assuming at level l when problem size = 1, then

$$\frac{n}{2^{l-1}} = 1 \Rightarrow l = \log_2(n) + 1$$

So for all i from 1 to $\log_2(n)$, the work done at level i is $2^{i-1}n$. At level $\log_2(n) + 1$, the work done at each node is 1, and there are $4^{\log_2(n)+1-1} = 4^{\log_2(n)} = 2^{\log_2(n)} \times 2^{\log_2(n)} = n \times n = n^2$ nodes, so total work at level $\log_2(n) + 1$ is n^2 .

$$\begin{aligned}
\text{Total cost} &= \left[\sum_{i=1}^{\log_2(n)} 2^{i-1} n \right] + n^2 \\
&= \frac{n}{2} \left[\sum_{i=1}^{\log_2(n)} 2^i \right] + n^2 \\
&= \frac{n}{2} \left[\frac{2^{\log_2(n)+1} - 1}{2 - 1} - 1 \right] + n^2 \\
&= \frac{n}{2} [2n - 1] + n^2 \\
&= 2n^2 - \frac{n}{2} \\
&\in O(n^2)
\end{aligned}$$

(b) (5 pts)

$$T(n) = \begin{cases} 1 & \text{if } n < 3 \\ T(\frac{n}{3}) + n & \text{if } n \geq 3 \end{cases}$$

Level	Problem Size	# Nodes	Work / Node	Level Total
1	$\frac{n}{3^0}$	1	n	n
2	$\frac{n}{3^1}$	1	$\frac{n}{3^1}$	$\frac{n}{3^1}$
...	...	...	...	...
i	$\frac{n}{3^{i-1}}$	1	$\frac{n}{3^{i-1}}$	$\frac{n}{3^{i-1}}$

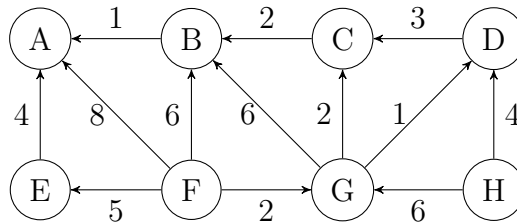
Assuming at level l when problem size = 3, then

$$\frac{n}{3^{l-1}} = 3 \Rightarrow l = \log_3(n)$$

So for all i from 1 to $\log_3(n)$, the work done at level i is $\frac{n}{3^{i-1}}$. At level $\log_3(n) + 1$, the work done at each node is 1, and there is 1 node, so total work at level $\log_3(n) + 1$ is 1.

$$\begin{aligned}
\text{Total cost} &= \left[\sum_{i=1}^{\log_3(n)} \frac{n}{3^{i-1}} \right] + 1 \\
&= n \cdot \left[\sum_{i=1}^{\log_3(n)} \frac{1}{3^{i-1}} \right] + 1 \\
&= n \cdot \left[\frac{-3 + 3n}{2n} \right] + 1 \quad [\text{Geometric Series}] \\
&= \frac{3}{2}n - \frac{1}{2} \\
&\in O(n)
\end{aligned}$$

3. (10 pts) Recall the Dijkstra's algorithm as presented in class. Consider the following weighted graph and write the valuation of d -value for each vertex at the end of each iteration of while-loop (that iterates until the priority queue is empty) in the algorithm. The source vertex is H.



(Add more rows to the following table, if needed, for your solution.)

Initial d -value	∞	∞	∞	∞	∞	∞	∞	0
Iteration	A	B	C	D	E	F	G	H
1	∞	∞	∞	4	∞	∞	6	0
2	∞	∞	7	4	∞	∞	6	0
3	∞	12	7	4	∞	∞	6	0
4	∞	9	7	4	∞	∞	6	0
5	10	9	7	4	∞	∞	6	0
6	10	9	7	4	∞	∞	6	0
7	10	9	7	4	∞	∞	6	0
8	10	9	7	4	∞	∞	6	0

The order in which the vertices are "added" to the priority queue are H, D, G, C, B, A. At the end of Dijkstra's algorithm, any vertex with an ∞ d -value is unreachable from the source H, in this case are vertices E and F.

4. (10 pts) Prove the following claim: for any cycle in a weighted graph with distinct **non-negative** edge weights, the edge with the highest weight in the cycle does not belong to a MST.

Proof by contradiction

Assume: $e = (u, v)$ be a highest weight edge in a cycle and it belongs to some MST T .

Not all edges in the cycle belongs to T (as T is a tree). Now, put the vertices reachable from u (without considering edge e) in graph in group A and put the vertices reachable from v (without considering edge e) in graph in group B . We have a cut of the graph (partition of vertices of the graph). Look at the cross-edges: e is a cross-edge. There should be some other cross-edges as well because e is part of a cycle. Furthermore, the edge e is not the min-wt cross-edge.

Therefore, it can be exchanged for a cross-edge that has smaller weight and create a new spanning tree that has a weight smaller than T .

Therefore, T is not an MST and our assumption is not valid.

5. (20 pts) Given two strings, write an algorithm for finding the longest common subsequence. A subsequence of a string is a sequence of characters which conforms to the relative ordering of the characters in the string. The characters do not necessarily

appear contiguously in the string.

Example: GHTCCHT and CHTGCH. The longest common subsequence is HTCH.

Design an algorithm using a Dynamic Programming strategy which consists of the following:

- (a) (**7 pts**) Formalize a recursive definition for the solution.
- (b) (**10 pts**) Write pseudocode for an iterative dynamic programming algorithm with a dictionary to implement the solution.
- (c) (**3 pts**) Analyze the runtime of your algorithm.

First, consider the optimization objective: find the length of the longest common subsequence. Second, formalize a recursive definition for the solution. Third, realize a dynamic programming algorithm to implement the solution (iterative algorithm with a dictionary). Finally, compute the longest common subsequence from the dictionary.

Objective: find the length of the longest common subsequence.

Consider a function which takes as input the two strings s and t containing m and n characters, respectively and outputs the length of the longest common subsequence. We will represent the function as $LCS(m, n)$. Therefore,

$$LCS(m, n) = \begin{cases} 0 & \text{if } m = 0 \vee n = 0 \\ 1 + LCS(m - 1, n - 1) & \text{if } s[m] == t[n] \\ \max\{LCS(m - 1, n), LCS(m, n - 1)\} & \text{otherwise} \end{cases}$$

The first case corresponds to the base case, when either one of the string is an empty string. The second case corresponds to the case when the last characters of the strings are equal. In the final case, the last characters of the strings are not the same. In this scenario, there are two possibilities to consider: (a) the last character of first string may match with the second last character of the second string or (b) the last character of the second string may match with the second last character of the first string.

The above recursive formulation shows the dependencies between the subproblems; the LCS for smaller values for m and n should be computed before the larger values. This results in the following iterative algorithm using DP strategy.

```
// dict[i][j]: holds the LCS(i, j) valuations
for i = 0 to m
  for j = 0 to n
    if i == 0 or j == 0
```

```

        dict[i][j] = 0
    else
        if s[i] == t[j] then
            dict[i][j] = 1 + dict[i-1][j-1]
        else
            dict[i][j] = max{ dict[i][j-1], dict[i-1][j] }

```

dict[m][n] holds the solution (Practice with two strings)
 Finding the longest common subsequence from the dictionary dict.

```

findLCS(i, j):
    if i == 0 or j == 0 return empty string
    if s[i] == t[j] return findLCS(i-1, j-1) + s[i];
    if dict[i][j] == dict[i-1][j]
        return findLCS(i-1, j)
    else
        return findLCS(i, j-1)

```

Invoke the above recursive function findLCS(m, n).
 Write the above iteratively.

```

// sol[i][j] = longest subsequence for strings of length i and j
for i = 0 to m
    for j = 0 to n
        if i == 0 or j == 0
            sol[i][j] = empty-string
        else
            if s[i] == t[j]
                sol[i][j] = sol[i-1][j-1] + s[i]
            else
                if dict[i][j] == dict[i][j-1]
                    sol[i][j] = sol[i][j-1]
                else
                    sol[i][j] = sol[i-1][j]

```

The construction of dict takes $O(m * n)$ time. The process of tracking the longest common subsequence also takes $O(m * n)$ time. Altogether, the runtime is $O(m * n)$.